MINOR PROJECT REPORT

CNN Image Classification and Streamlit Application

MNIST Handwritten Digit Classification and a Student Marks Calculator App

Student Name	Karthik Somaraju
Batch	A
Submission Date	02.11.2026
Project Type	CNN Image Classification and Streamlit Application (Individual Project)
Selected Dataset	MNIST (handwritten digits 0 to 9, 70,000 images)
Streamlit App Topic	Student Marks Calculator
Programme	Artificial Intelligence Program

2. Objective

This project applies a Convolutional Neural Network (CNN) to an image classification problem and builds a simple, usable Streamlit web application. The aim is to understand the complete workflow, evaluate the results honestly and explain the work clearly.

- **Part A - CNN:** train a CNN that looks at a 28 x 28 grayscale image of a handwritten digit and predicts which digit (0 to 9) it is. This is a 10-class image classification problem, and it is measured by accuracy on unseen test images.
- **Part B - Streamlit:** build a Student Marks Calculator where the user enters a student name and marks for five subjects and instantly gets the total, percentage, grade and a pass / fail result.

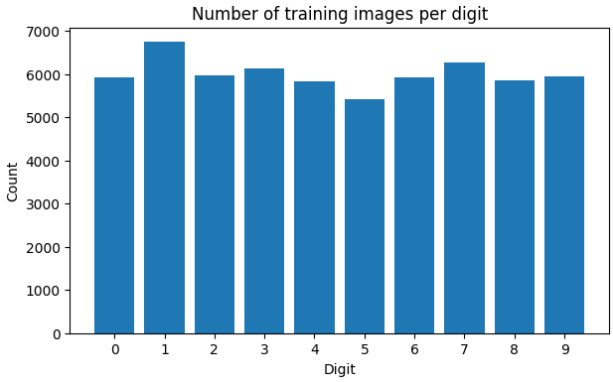
3. CNN Workflow

3.1 Dataset Description and Class Names

MNIST is a standard benchmark of handwritten digits and is loaded directly from Keras, so no file upload is needed.

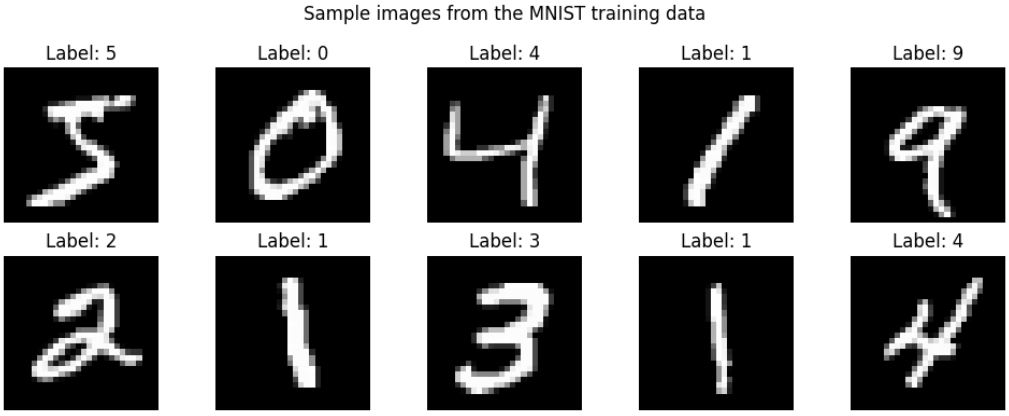
Property	Value
Total images	70,000 (60,000 training + 10,000 test)
Image size	28 x 28 pixels, grayscale (1 channel)
Pixel values (raw)	0 to 255 (integers)
Number of classes	10
Class names	0, 1, 2, 3, 4, 5, 6, 7, 8, 9
Array shapes	x_train (60000, 28, 28), y_train (60000,), x_test (10000, 28, 28), y_test (10000,)

Chart 1: Number of Training Images per Digit



Observation: every digit has roughly the same number of images (from 5,421 images of digit 5 to 6,742 images of digit 1), so the dataset is balanced and accuracy is a fair measure of performance.

Chart 2: Sample Images from the Training Data



Observation: the digits are white strokes on a black background and vary a lot in thickness, slant and style, which is what makes the task non-trivial.

3.2 Preprocessing

Step	What was done	Why
Normalization	Pixel values divided by 255.0 (range 0 to 255 becomes 0.0 to 1.0)	Small, similar-scale inputs help the network train faster and more stably
Channel dimension	Reshaped from (28, 28) to (28, 28, 1)	A Conv2D layer expects (height, width, channels)
Labels	Kept as integers 0 to 9 (no one-hot encoding)	sparse_categorical_crossentropy works directly with integer labels

3.3 CNN Approach and Model Summary

A normal neural network treats an image as a long list of numbers and loses its shape. A CNN keeps the shape: convolution filters slide over the image to find patterns such as edges and curves, pooling shrinks the image, and dense layers use the patterns to decide the digit.

Layer	Output Shape	Parameters	What it does
Conv2D (32 filters, 3 x 3, ReLU)	(26, 26, 32)	320	Finds edges and curves
MaxPooling2D (2 x 2)	(13, 13, 32)	0	Keeps the strongest value in each 2 x 2 block, halving the size
Flatten	(5408)	0	Turns the 2D result into one long list
Dense (64, ReLU)	(64)	346,176	Combines the patterns
Dense (10, softmax)	(10)	650	One probability per digit
Total		**347,146**	All parameters are trainable

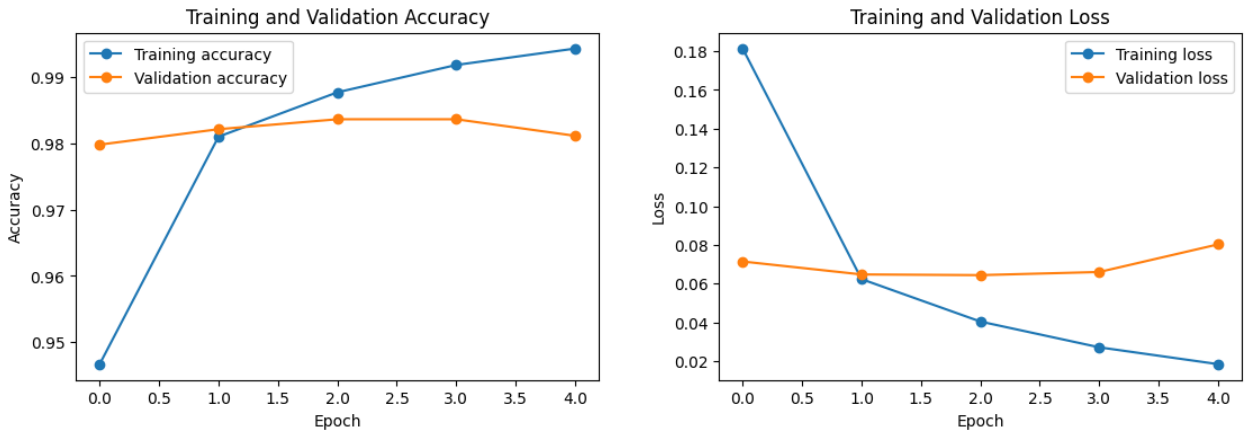
The model was compiled with the Adam optimizer, sparse categorical cross-entropy loss and accuracy as the metric.

3.4 Training

The model was trained for 5 epochs. 10% of the training data (6,000 images) was kept aside as a validation set, so the model could be checked on unseen images after every epoch.

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss
1	0.9467	0.1814	0.9798	0.0715
2	0.9810	0.0624	0.9822	0.0648
3	0.9877	0.0405	0.9837	0.0644
4	0.9919	0.0272	0.9837	0.0661
5	0.9944	0.0185	0.9812	0.0803

Chart 3 and Chart 4: Training and Validation Accuracy / Loss

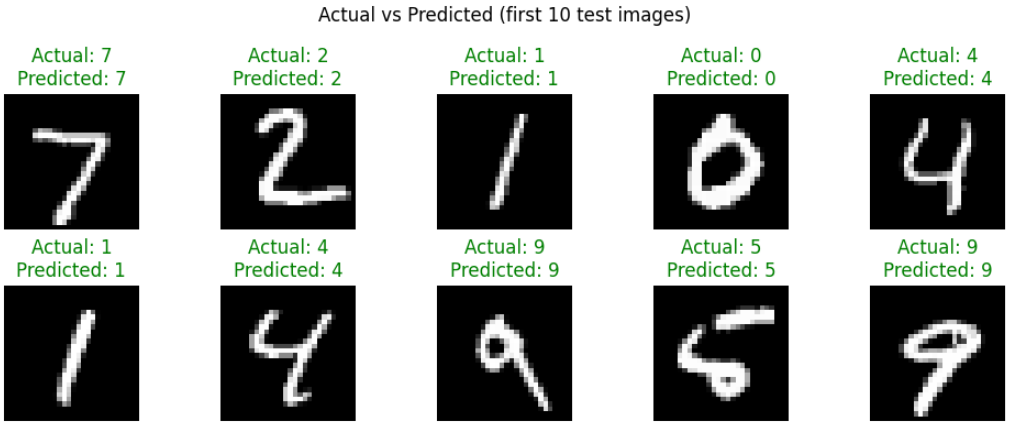


Observation: training accuracy keeps rising and training loss keeps falling every epoch. Validation accuracy levels off at about 98.4% after epoch 3, and validation loss starts to rise from epoch 4 (0.0644 to 0.0803). This shows the model is beginning to memorise the training data a little (mild overfitting), so training for more epochs would not help.

3.5 Evaluation on Test Data

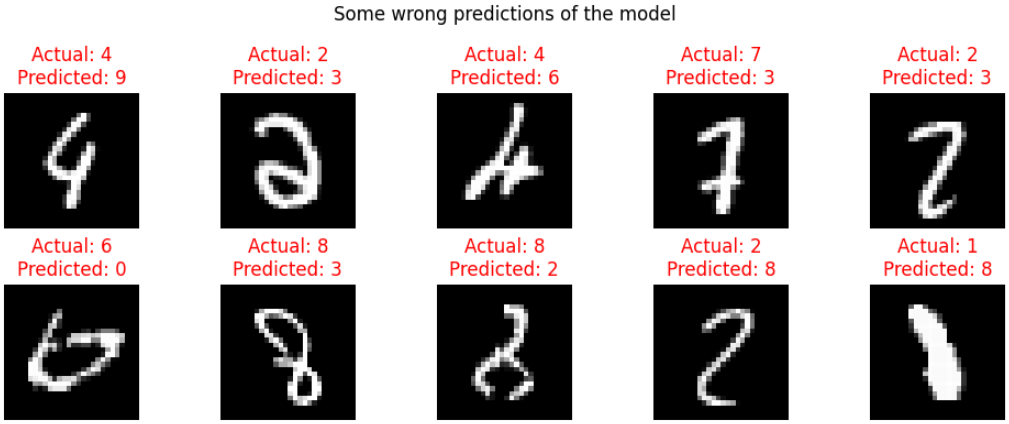
Metric	Value
Test loss	0.0683
Test accuracy	0.9794 (97.94%)
Correct predictions	9,794 out of 10,000
Incorrect predictions	206 out of 10,000
Final training / validation accuracy	99.44% / 98.12%

Chart 5: Actual vs Predicted Labels (first 10 test images)



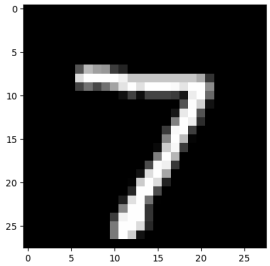
Observation: all 10 of the first test images are predicted correctly (green titles).

Chart 6: Some Wrong Predictions of the Model



Observation: the wrong predictions are mostly messy or unusual handwriting, for example a 4 written like a 9, a 2 with a closed loop that looks like a 3, and a very thick 1 predicted as an 8. Even a person may find several of these confusing.

Prediction probabilities for one image



The first test image (a handwritten 7) was passed to the model. It predicted **digit 7 with 100.00% probability** and 0.00% for every other digit, so the softmax output is very confident for clear images.

Chart 7: Single image prediction

4. Streamlit App: Student Marks Calculator

Purpose: quickly calculate a student's result from the marks of five subjects (Mathematics, Science, English, Social Studies and Computer Science), each out of 100. The app is a single Python script (*app.py*) run with *streamlit run app.py*.

Item	Details
User inputs	Student name (text box) and marks for five subjects (number boxes, whole numbers from 0 to 100), then the Calculate Result button
Processing	Total = sum of marks; Percentage = total / 500 x 100; Grade from percentage; a subject is failed if marks are below 35
Grades	A+ (90 and above), A (80-89), B (70-79), C (60-69), D (50-59), F (below 50)
Outputs	Total out of 500, percentage, grade, PASS / FAIL message listing subjects that need improvement, and a bar chart of the marks
Invalid input handling	Empty name shows an error; a name with digits or symbols shows an error; all marks equal to 0 shows a warning; the marks box only accepts values from 0 to 100
Layout	Title and instructions at the top, inputs in two columns, results shown in three metric boxes followed by the chart

Sample runs (calculated with the app's own logic)

Input marks (Maths, Science, English, Social, CS)	Total	Percentage	Grade	Result
85, 78, 92, 67, 88	410 / 500	82.0 %	A	PASS
45, 30, 60, 52, 40	227 / 500	45.4 %	F	FAIL - needs improvement in Science

Working screenshots

Screenshot 1: app input page

Screenshot 2: app result page

5. Results

- **Model performance:** the CNN reached 97.94% test accuracy (9,794 of 10,000 images correct) with a test loss of 0.0683. Training, validation and test accuracy (99.44%, 98.12%, 97.94%) are close, so the model generalises well to unseen images.
- **Predictions:** clear digits are classified with very high confidence; the 206 errors are mostly unclear or unusual handwriting.
- **App output:** the Student Marks Calculator returns the total, percentage, grade and pass / fail result instantly, and handles empty names, invalid names and all-zero marks without crashing.
- **Important finding:** validation loss started to rise after epoch 3 while training loss kept falling, a sign of mild overfitting even in a small model.

6. Conclusion

Learning: this project covered the full CNN workflow - loading data, checking shape and class balance, normalizing, adding the channel dimension, building, training, evaluating and explaining predictions - and how to turn a Python script into a working web app with Streamlit.

Challenges: understanding the required input shape for Conv2D, choosing the correct loss for integer labels, reading the training curves to spot overfitting, and validating user input in the app.

Possible improvements: add a second Conv + Pooling block and Dropout; use EarlyStopping so training stops at the best epoch; add data augmentation (small rotations and shifts); add a confusion matrix and classification report; and extend the Streamlit app so it loads the trained CNN and recognises an uploaded digit image.